

Shared Memory

Objectives:

- Understand processes and `fork()` in Linux.
- Learn to use `wait()` and `waitpid()` to manage child processes.
- Explore `exec()` system call to run new programs.

Shared Memory:

Shared memory is a memory management technique used in operating systems that allows multiple processes to access a common memory space. This shared memory region enables fast data communication between processes since they can directly read and write to this shared area without needing to use slower inter-process communication (IPC) methods like pipes or message queues.

How It Works

- When a process wants to share data, it requests a block of memory from the operating system to be designated as "shared."
- This shared block is then available to other processes, which can also request to connect to this block.
- Once connected, processes can read from and write to this memory block as if it's a part of their own memory space.

Creating Shared Memory

- In most operating systems, shared memory is created using system functions. For example, in UNIX-based systems, the function `shmget()` creates a shared memory segment.
- This segment gets a unique ID, kind of like a Wi-Fi password that's shared among all the processes that need access.

Connecting to Shared Memory

- Each process that wants to access this shared space must "attach" to it. In UNIX, they do this with `shmat()`, which means "shared memory attach."
- When a process attaches to the shared memory, the OS connects this shared area to the process's memory, letting it use the shared memory like any other part of its memory.

Reading and Writing to Shared Memory

- Once connected, each process can directly read from and write to this memory.
- For instance, one process can write data to this memory space, and the other can immediately read it—just like one person writing on a whiteboard and another reading from it right away.

The Need for Synchronization

- Because multiple processes can access shared memory at once, they can sometimes get in each other's way if they try to read or write at the same time. This is called a *race condition*.
- **Example:** Imagine two people trying to write on the same part of a whiteboard at the same time; their messages would get mixed up.
- To prevent this, processes use synchronization tools like **mutexes** (locks) or **semaphores** to coordinate access.

Removing Shared Memory

- When processes are done using the shared memory, they detach from it, similar to disconnecting from a network.
- Finally, the operating system can remove the shared memory block, freeing up the memory for other uses.

Real-Life Example of Shared Memory

- **Database Systems:** In database applications, many processes may need access to the same data, like user records. Instead of copying data between them, they can share a memory area that holds this data. This speeds up database transactions since all processes can look at the same data in real-time.
- **Graphics Rendering:** In multimedia applications, such as video games or 3D rendering software, shared memory allows fast communication between processes that handle different parts of the graphics pipeline. This is crucial for real-time performance.

Functions

1. **ftok**

The `ftok` function in UNIX-like operating systems is used to generate a unique key (identifier) that multiple processes can use to access the same shared memory segment.

```
key_t key = ftok(char *filename, 0);
```

2. **shmget**

```
int id = shmget(key_t key, int size, int flags);
```

```
// int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
```

- Allocates a shared memory segment.
- Key is the key associated with the shared memory segment you want.
- Size is the size in bytes of the shared memory segment you want allocated.
- Memory is allocated in pages, so chances are you will probably get a little more memory than you wanted.
- Flags indicate how you want the segment created and its access permissions.
- The general rule is just to use `0666 | IPC_CREAT | IPC_EXCL` if the caller is making a new segment.
- If the caller wants to use an existing share region, simply pass 0 in the flag.

RETURN VALUES

- Upon successful completion, `shmget()` returns the positive integer identifier of a shared memory segment. Otherwise, -1 is returned

Shmget will fail if:

1. Size specified is greater than the size of the previously existing segment. Size specified is less than the system imposed minimum, or greater than the system-imposed maximum.
2. No shared memory segment was found matching the key, and `IPC_CREAT` was not specified.
3. The kernel was unable to allocate enough memory to satisfy the request.

4. IPC_CREAT and IPC_EXCL were specified, and a shared memory segment corresponding to the key already exists.

3. shmat

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

- Maps a shared memory segment onto your process's address space.
- shmid is the id as returned by shmget() of the shared memory segment you wish to attach.
- Addr is the address where you want to attach the shared memory. For simplicity, we will pass NULL.
- NULL means that the kernel itself will decide where to attach it to address space of the process.

RETURN VALUES

- Upon success, shmat() returns the address where the segment is attached;
- Otherwise, -1 is returned and errno is set to indicate the error.

Shmat() will fail if:

1. No shared memory segment was found corresponding to the given id.

4. shmdt

```
int shmdt(void *addr);
```

- This system call is used to detach a shared memory region from the process's address space.
- Addr is the address of the shared memory

RETURN VALUES

On success, shmdt() returns 0; on error -1 is returned

shmdt will fail if:

1. The address passed to it does not correspond to a shared region.

5. shmctl

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

- Delete shared memory region
- shmctl(shmid, IPC_RMID, NULL);
- shmctl() performs the control operation specified by cmd on the System V shared memory segment whose identifier is given in shmid.
- For Deletion, we will use IPC_RMID flag.
- IPC_RMID marks the segment to be destroyed.
- The segment will actually be destroyed only after the last process detaches it (The caller must be the owner or creator of the segment, or be privileged).
- The buf argument is ignored.

Return Value:

For IPC_RMID operation, 0 is returned on success; else -1 is returned.

Example:

Process 1:

- Creates a shared memory area.

- Writes text (passed as a command-line argument) to the shared memory.
- Waits for 10 seconds to allow Process 2 to read the data.
- Unlinks and deletes the shared memory area after 10 seconds.

Process 2:

- Attaches to the shared memory area created by Process 1.
- Reads the text from shared memory and prints it.
- Unlinks itself from the shared memory.

Process1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024 //Size of shared memory, value can be directly passed

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <text_to_send>\n", argv[0]);
        exit(1);
    }

    // Generate a unique key for shared memory
    key_t key = ftok("shmfile", 65);

    // Create a shared memory segment
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    if (shmid == -1) {
        perror("shmget");
        exit(1);
    }

    // Attach to the shared memory
    char *data = (char *) shmat(shmid, NULL, 0);
    if (data == (char *)(-1)) {
        perror("shmat");
        exit(1);
    }

    // Write text to shared memory
    strncpy(data, argv[1], SHM_SIZE);
    printf("Process 1: Wrote '%s' to shared memory\n", data);

    // Wait for 10 seconds before unlinking and deleting shared memory
    sleep(10);

    // Detach from shared memory
    if (shmdt(data) == -1) {
        perror("shmdt");
    }
}
```

```

    exit(1);
}

// Delete the shared memory segment
if (shmctl(shmid, IPC_RMID, NULL) == -1) {
    perror("shmctl");
    exit(1);
}

printf("Process 1: Unlinked and deleted shared memory\n");

return 0;
}

```

Process2:

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024 // Size of shared memory

int main() {
    // Generate a unique key for shared memory (must match Process 1)
    key_t key = ftok("shmfile", 65);

    // Access the shared memory segment
    int shmid = shmget(key, SHM_SIZE, 0666);
    if (shmid == -1) {
        perror("shmget");
        exit(1);
    }

    // Attach to the shared memory
    char *data = (char *) shmat(shmid, NULL, 0);
    if (data == (char *)(-1)) {
        perror("shmat");
        exit(1);
    }

    // Read and print the text from shared memory
    printf("Process 2: Read from shared memory: '%s'\n", data);

    // Detach from the shared memory
    if (shmdt(data) == -1) {
        perror("shmdt");
        exit(1);
    }

    printf("Process 2: Detached from shared memory\n");

    return 0;}

```

Open two terminals and run both processes simultaneously.

Output:

```
fatima@fatima-VMware-Virtual-Platform:~/Desktop$ gcc -o p1 p1.c
fatima@fatima-VMware-Virtual-Platform:~/Desktop$ ./p1 "Hello from p1"
wrote data Hello from p1
fatima@fatima-VMware-Virtual-Platform:~/Desktop$
```

```
fatima@fatima-VMware-Virtual-Platform:~/Desktop$ gcc -o p2 p2.c
fatima@fatima-VMware-Virtual-Platform:~/Desktop$ ./p2
read data Hello from p1
fatima@fatima-VMware-Virtual-Platform:~/Desktop$
```